INFORMATION BOUNDS AND EXACT INDICES FOR ORDERED FLOATING-POINT ADDITION

ARTEM D. POPKOV

Abstract. A block of sequentially rounded additions defines a function of its incoming accumulator. We study the worst-case number of bits needed to describe this function before the incoming value is known. At significand precision $p \geq 4$, an explicit family of blocks with $2k$ additions realizes $\binom{2^{p-2}-2}{k}$ distinct functions, even though each block has real sum zero and returns zero from an initial zero. This gives optimal-order $\Theta(mp)$ descriptions for $2 \leq m \leq 2^{\alpha p}$, for each fixed $0 < \alpha < 1$, when an operand occupies $\Theta(p)$ bits. For $p \geq 6$, on a common query grid whose rounded trajectories remain within two adjacent positive binades, we obtain the matching bound $\Theta\big(m \log_2(1 + 2^p/m)\big)$ for all $m \geq 2$. Its upper bound follows from a residue potential: adjacent output gaps start at four and have total deviation at most $4m$. A gcd potential extends this order on the same grid to any fixed number K of adjacent positive binades, with constants depending on K. A two-threshold normal form for uniform dyadic rounding compositions gives a different exact index with at most $1 + (K-1)m$ pieces. The same construction covers every finite input of a binary format with E normal binades, including gradual underflow and absorbing overflow, using at most $2E + 1 + 2Em$ pieces and $O(\log(E(m+1)+2))$ abstract operations per query. It identifies signed zeros numerically and excludes NaNs and nonfinite operands. Its construction does not require an initial full-state table. A second family uses permutations of one public multiset of $4k$ distinct operands to realize $k!$ functions when $2^p \geq 64k^2$. The asymptotic lower bounds persist under any fixed pointwise error probability below one half. Exact arithmetic and native hardware checks support the constructions. These are bounds on standalone one-way descriptions.

1. Introduction

When rounding is required after each specified addition, summation is an ordered computation. A block of additions can nevertheless be viewed as a function of its incoming accumulator. Preparing this function before the preceding block finishes is one possible route to processing blocks in parallel. Finite-state transition composition provides such a representation

Date: November 7, 2026.

2020 *Mathematics Subject Classification.* Primary 65G50, 68Q11; Secondary 68P30.

Key words and phrases. Floating-point arithmetic, ordered accumulation, transfer function, communication complexity, succinct representation.

in principle [15, 19]. We ask how much information a short block of rounded additions can place in that function.

The encoder knows an ordered operand block $a = (a_1, \dots, a_m)$ but does not know the incoming accumulator x. It prepares a standalone message from which a decoder must recover the exact sequentially rounded result at any allowed x. Encoding and decoding time are unrestricted. The resulting lower bounds therefore apply to tables, compressed data structures, circuits, and any other representation obeying this access contract. Sending the operands and replaying the block is an elementary upper bound.

This contract distinguishes a summary from the usual sequential computation. Once x is known, one accumulator suffices to evaluate the block. With interaction, the decoder can send x to the block owner and receive the answer. A description prepared without x must instead preserve the behavior of the block over the entire query set. We study the information required by that description, without asserting that ordered addition is inherently sequential.

A lower bound requires arithmetic attainability. Counting all functions on a finite set does not establish that rounded additions realize many of them in a bounded number of operations. Conversely, monotonicity alone allows maps that need not be generated by a short operand block. We address both sides of this distinction.

Section 4 constructs an injective encoding of threshold subsets, using two additions per selected threshold. This gives a $\Theta(mp)$ description bound for sufficiently short blocks at precision p, when an operand occupies $\Theta(p)$ bits. In Section 5, a potential on adjacent output gaps gives the matching bound

$$\Theta\left(m \log_2 \left(1 + \frac{2^p}{m}\right)\right)$$

on a common query grid under a two-binade trajectory restriction. The same argument yields a sparse evaluator and a construction algorithm that does not enumerate the input grid. Section 6 extends the potential to every fixed number of adjacent positive binades, using residue classes of the original grid. Section 7 removes the exact-sum restriction and gives a different evaluator whose record count is polynomial in the number of binades. Section 8 extends that evaluator to the entire finite format, including negative inputs, subnormals, and absorbing overflow. Section 9 holds the numerical multiset fixed and encodes permutations, isolating the information carried by operand order. Section 10 extends the lower bounds to pointwise bounded error by standard entropy arguments.

The matching information bounds retain a trajectory restriction: rounded states must remain in the specified positive corridor for every allowed query. We first prove the bounds under a stronger exact-sum promise and then remove that extra condition. The whole-format evaluator needs no corridor promise, but has a different, generally larger size bound. Construction from

operands and query evaluation are established; an efficient merge algorithm on two constructed indices is not claimed.

2. Related work

2.1. **Transition summaries and ordered accumulation.** Ladner and Fischer compose input-induced maps in a finite-state construction [15]. Mytkowicz et al. enumerate entry states and exploit coinciding images and their organization into ranges [19, Sections 3 and 5]. Brandoit et al. formulate finite-bit scan representations and the corresponding cardinality ceiling [2, Section 5 and Appendices F.4 and G.2]. Our interface requires only evaluation at the eventual incoming state, and is weaker than requiring a closed exact merge operation. Transition composition and the message-counting principle are therefore established tools. The arithmetic question is how many maps a bounded number of rounded additions can attain.

Kapre and DeHon compute the sequentially specified sum using optimistic prefixes, recurrence checks, and iterative correction [12, Section III]. Their method retains the original data and performs further work after detecting errors. It falls outside the standalone description contract and is not excluded by our bounds. More generally, nonassociativity alone does not imply a parallel lower bound: saturated accumulation admits compact associative reformulations [22].

2.2. **Rounded translations and their compositions.** The rounded translate-and-undo operation is classical. Under stable nearest rounding, Reiser and Knuth prove that

$$T_y(x) = \mathrm{RN}(\mathrm{RN}(x+y)-y) \quad \text{satisfies} \quad T_y(T_y(x)) = T_y(x)$$

whenever the operations are defined [23, p. 86, Theorem 1]; the corrections in [7] are incorporated in the later reprint [24]. Their stable class permits a consistent choice of parity for each exponent and sign and is broader than a single global ties-to-even rule. Their second theorem concerns the intervening addition outputs, which need not already be fixed. Goldberg reproduces the fixed-offset result for ties-to-even [10, Theorem 5]. Voráček and Hein use related finite-precision structure in sound randomized smoothing and cite the earlier result [27, Section 3.1].

The local parity-sensitive cancellation in our construction is a specialization of this classical arithmetic. The exponent-alignment and midpoint cases of Reiser and Knuth [23, pp. 85–86] account for its individual pairs. Our lower bound uses an ordered composition of pairs with varying offsets and alternating parity to encode a threshold subset. Section 9 adds exact surrounding translations and disjoint threshold bands to encode permutations of one public numerical multiset. On the witness states, a four-operation slot is itself a rounded translation T_{s+r}; its role is to implement the assignment of thresholds while preserving the multiset. Neither the primitive nor its idempotence is a new claim.

The addendum to the reprint points to a subsequent result of Diamond [24, p. 348]. For a fixed continuous, strictly monotone function f that is strictly convex or strictly concave, Diamond proves $h^3 = h^2$ on representable inputs for

$$h = \mathrm{RN} \circ f^{-1} \circ \mathrm{RN} \circ f,$$

provided the radix raised to the precision is at least three and all iterates are defined [5, pp. 228–232]. Nearest-rounding ties may be chosen arbitrarily. The proof uses rounding cells and secant slopes along this linked orbit. Our upper bound instead compares independently initialized trajectories under varying additions and sums their absolute gap changes over both inputs and time. Affine translations also lack Diamond's strict curvature hypothesis. The addendum's multiplication/division pointer concerns another fixed forward/inverse pair: Knuth's exercise 4.2.2–22 and its answer show that $M_v(x) = \mathrm{RN}(\mathrm{RN}(xv)/v)$ satisfies $M_v^2 = M_v$ on representable inputs x, for fixed nonzero representable v, provided no intermediate operation underflows or overflows [13, pp. 229 and 573]. The third edition retains this result with an amended boundary-case argument [14, pp. 244 and 616]. Neither version counts the transfer maps attainable by varying blocks of additions or bounds their description size.

Casacuberta et al. establish order sensitivity and repeated-rounding effects in differential-privacy implementations [3, Sections 5.3, 5.5, and 5.6]. FPRev infers an unknown accumulation tree by changing the placement of large positive and negative masks among its inputs [28]; its basic probes can preserve the multiset. In our interface, the operands and their order in each block stay fixed while only the incoming scalar varies. Monroe and Job count structurally inequivalent labeled summation forms [18, Sections 2.1 and 6.1, Proposition 27]. That count does not establish distinct one-variable transfers for one fixed numerical multiset.

Nowak, Cotterell, and Boumasmoud study finite-precision transition monoids and their aperiodicity [20, Appendix J]. Their subsequent transformer analysis realizes reset behavior on a restricted state action through a reset-monoid divisor [21, Appendix C]. These algebraic properties are compatible with many distinct length-bounded maps. In particular, the existence of a rounded reset does not identify the separated query markers in our attainable families.

2.3. **Compression and information bounds.** Breakpoint propagation under monotone composition is classical [8, Lemma 2.1]. Gap coding, subtraction of a known spacing, and succinct representations of ordered positions are also established techniques [26]. We prove a linear bound on the total magnitude of signed gap deviations. Combined with attainability on the same promised query domain, it gives the matching description bound. The support estimate and sparse constructor follow from spacing-four invariance away from a single boundary crossing.

Generic binary rounding theory, including representability and the ordering of lattice rounding, is formalized in libraries such as FloatLib, building on Flocq [9, Section 2.3]. The uniform-lattice facts used below are elementary ingredients of that theory. Conditions for simplifying consecutive rounding operations are a different question [25, Section 5.2 and Appendix A.2]: the gap argument retains every prescribed rounding and compares two trajectories. Its additional statement is a budget for their total gap variation, charged to crossings of binade boundaries.

The bounded-error extensions use standard augmented-index and entropy arguments [4]. We include the entropy proof to distinguish that general inequality from the arithmetic query reductions. The contribution lies in the attainable families and the aggregate gap bound, rather than in a new communication inequality or a new rounded cancellation primitive.

2.4. **Staircase representations.** Eggleton classifies fixed insertion patterns of floors and ceilings in a linear expression [6]; Lagarias et al. classify commuting dilated floor functions [16]. Our normal form instead allows arbitrarily long translated nearest-even chains with varying dyadic spacings. Its proof combines translation equivariance with a common output coset to leave at most two thresholds in one period.

Monotone piecewise-constant lists and their operations are established data structures. Henzinger et al. control their size through approximation [11, Section 2]. Piecewise symbolic floating-point models can also deliberately overapproximate individual roundings to verify programs [17, Sections 4.2–4.3]. Bagnara et al. treat IEEE special values and optimal or sound interval projections in detail [1, Sections 1.4 and 4.1–4.3]. These contracts differ from preserving the entire input-output graph of a fixed operand chain. The index below is exact on every permitted scalar input. Its structural ingredients are the two-threshold closure and the global count of preimage cuts, rather than interval lists or IEEE case distinctions themselves.

3. Model and notation

All logarithms and information quantities use base two. We write $\binom{n}{k}$ for a binomial coefficient.

Fix a binary floating-point format with p significant bits, including the leading bit, and round-to-nearest with ties to even. Let RN denote correct rounding. Assume that the finite normal exponent range contains exponents 0 through p, with the exponent defined by the normalized form $(1.f)_2 \times 2^e$. Set $R = 2^p$. In the corridor

$$C = [R/2, 2R),$$

the lower binade has spacing one and the upper binade has spacing two. Every representable state in C is an integer. The witness trajectories avoid overflow, underflow, NaNs, and distinctions between signed zeros. The decoder returns the numerical result; floating-point exception flags are outside the contract.

For a block $a = (a_1, \dots, a_m)$ of finite representable operands, define

$$s_0 = x, \qquad s_i = \mathrm{RN}(s_{i-1} + a_i) \quad (1 \le i \le m), \qquad F_a(x) = s_m. \tag{3.1}$$

The format, block length, and allowed query set X are public. An encoder E depends only on a. A decoder D must satisfy

$$D(E(a), x) = F_a(x) \qquad \text{for every allowed block } a \text{ and every } x \in X. \tag{3.2}$$

The message contains all block-specific information available to D. A pointer therefore does not constitute a short standalone summary if it grants access to uncounted operands or auxiliary data. Public constants and algorithms independent of the chosen block are free.

Let $B(\mathcal{A}, X)$ be the minimum worst-case fixed message length for a block class $\mathcal{A}$ on X. Encoding and decoding are computationally unrestricted. If the class realizes M distinct functions on X, then

$$B(\mathcal{A}, X) \ge \lceil \log_2 M \rceil, \tag{3.3}$$

since different functions cannot share a message. Conversely, an index into the finite transfer family is an information-theoretic upper bound. These standard facts depend on the message convention. With arbitrary strings of length at most b and free external framing, there are $2^{b+1} - 1$ messages, giving the corresponding lower bound $\lceil \log_2(M + 1) \rceil - 1$.

For bounded-error protocols, public random coins U, independent of the input, may affect both encoding and decoding. For each fixed pair (a, x), the probability of returning the exact result must be at least $1 - \varepsilon$, where $0 \le \varepsilon < 1/2$. This is a pointwise guarantee; it does not require one random message to answer every query correctly at once. Making private coins public only strengthens a protocol, so our lower bounds also apply to private randomness.

The corridor class $\mathcal{A}(p, m, X)$ consists of blocks for which both $s_{i-1} + a_i$ and s_i belong to C at every step and for every $x \in X$. The encoder is required to describe only blocks satisfying this promise. The reference constructor also detects violations by checking the extreme trajectories.

4. An attainable family of rounded-add transfers

4.1. **Threshold construction.**

Theorem 4.1 (Binomial attainability)**.** *Let $p \geq 4$, put*

$$X_0 = \{8, 12, \ldots, R-4\}, \qquad N = R/4 - 2,$$

and let $0 \leq k \leq \min\{N, \lfloor m/2 \rfloor\}$ be an integer. Blocks of length m realize at least $\binom{N}{k}$ distinct transfer functions on X_0. Every block in the constructed family has exact real sum zero and satisfies $F_a(0) = 0$. On X_0, all its intermediate values are finite, normal, and positive. Consequently, every exact fixed-width description for a class containing this family requires at least $\lceil \log_2 \binom{N}{k} \rceil$ bits.

Proof. Choose a k-element subset $S \subseteq X_0$ and write its elements as $t_0 < \cdots < t_{k-1}$. Define

$$c_j = \begin{cases} 1, & j \text{ even}, \\ 4, & j \text{ odd}, \end{cases} \qquad r_j = R + c_j - t_j. \tag{4.1}$$

Concatenate the pairs $(+r_j, -r_j)$ in increasing order of j and append $m - 2k$ positive zeros. Each r_j is an integer strictly between zero and R, so every operand is representable. Every pair has exact real sum zero and returns exactly to zero when started from zero.

After any prefix of pairs, we show that the state at a marker $x \in X_0$ is $x - d$, where d is the parity of the number of processed thresholds not exceeding x. Before pair j, every marker $x \geq t_j$ has crossed all j preceding thresholds, and hence $d = j \bmod 2$. A marker below t_j may have either parity.

If $x < t_j$, the spacing-four grid gives $x \leq t_j - 4$. The exact first sum is the positive integer

$$x - d + R + c_j - t_j \leq R.$$

The subsequent exact difference is $x - d \in [7, R)$. Both operations are exact, so the state is unchanged.

If $x \geq t_j$ and j is even, then $d = 0$. The exact first sum is $R + 1 + (x - t_j)$, which lies in $[R, 2R)$ and is congruent to one modulo four. Round-to-nearest with ties to even decreases it by one. Exact subtraction of r_j then gives $x - 1$. If j is odd, then $d = 1$; the exact first sum is $R + 3 + (x - t_j)$ and is congruent to three modulo four. It rounds up by one, and subtracting r_j gives x. The exact sums and their rounded values remain strictly below $2R$. Thus the completed block has transfer function

$$F_S(x) = x - \big(|S \cap (-\infty, x]| \bmod 2\big). \tag{4.2}$$

The first threshold in the symmetric difference of two subsets distinguishes their transfer functions. Equivalently, adjacent differences of the bits $x - F_S(x)$ recover membership in S, with initial parity zero before the first marker. Hence all $\binom{N}{k}$ transfers are distinct. Padding by positive zero preserves their values. The fixed-width counting bound completes the proof. □

Adjacent markers differ by four, whereas each final output differs from its marker by at most one. Their outputs are therefore distinct. Since deterministic trajectories cannot separate after they have coalesced, no two marker trajectories in this construction coalesce at an earlier step. Merging coincident images cannot reduce this family.

Corollary 4.2 (Short-block incompressibility)**.** *Fix $0 < \alpha < 1$. As p tends to infinity, exact summaries require $\Omega(mp)$ bits, uniformly for $2 \le m \le 2^{\alpha p}$. If an operand word has width $w = \Theta(p)$, the optimal worst-case description length is $\Theta(mp)$.*

Proof. For sufficiently large p, take $k = \lfloor m/2 \rfloor \ge m/3$ and use $N \ge 2^{p-3}$. The bound $\binom{N}{k} \ge (N/k)^k$ gives

$$\log_2 \binom{N}{k} \ge k((1-\alpha)p - 3).$$

The lower-bound constant may depend on α. Sending all m operand words gives the upper bound mw. □

For longer blocks, the choice $k = \min\{\lfloor m/2 \rfloor, \lfloor N/2 \rfloor\}$ gives a lower bound of order $m \log_2(1 + R/m)$, which saturates at order R. In particular, the bound is not linear in m when the format is fixed and m tends to infinity. With $m = 256$, the exact binomial expression gives lower bounds of 410, 2100, and 5812 bits at precisions $p = 11$, 24, and 53, respectively.

4.2. **The same construction in a narrow corridor.** For $p \ge 6$, define

$$X_* = \{R/2 + 8, R/2 + 12, \dots, 3R/4 - 4\}, \qquad N_* = R/16 - 2. \tag{4.3}$$

Restrict both thresholds and query markers to X_*. The preceding proof still applies. Operand magnitudes lie in $[R/4, R/2)$, every state at a pair boundary is at least $R/2 + 7$, and every exact sum and rounded intermediate value stays in $C = [R/2, 2R)$. The family therefore belongs to $\mathcal{A}(p, m, X_*)$, on the same query set for every block. It has $\binom{N_*}{k}$ distinct transfers. Since $N_* = \Theta(R)$, its cardinality has the same asymptotic growth as that of the wider-grid family.

5. A matching upper bound in two adjacent binades

5.1. **Local gap dynamics.** Let $x_0, \dots, x_{L-1}$ be an increasing arithmetic progression of representable integers with spacing four. Consider a block satisfying the corridor promise on the entire progression. After a prefix of the block, write

$$y_i = F(x_i), \qquad g_i = y_{i+1} - y_i \quad (0 \le i < L - 1).$$

Correct rounding is monotone, so the gaps are nonnegative integers. Initially every gap equals four. Adding a common operand preserves each exact gap before rounding.

Lemma 5.1 (Local weighted bound)**.** *For a nonnegative integer g, define*

$$\psi(g) = \begin{cases} 0, & g \equiv 0 \pmod 4, \\ 2, & g \equiv 2 \pmod 4, \\ 3, & g \text{ odd}. \end{cases} \tag{5.1}$$

Suppose one rounded addition changes a gap from g to h and obeys the corridor promise at both endpoints. If the exact endpoints do not strictly straddle R, then

$$|h - g| + \psi(h) - \psi(g) \le 0. \tag{5.2}$$

If they strictly straddle R, the left-hand side is at most four. A gap equal to four is unchanged unless it strictly straddles R.

Proof. Below R, the rounding lattice has spacing one. Two exact values at an even integer distance round to values at the same distance. For an odd distance, the gap can change only at a half-integer tie. The new gap is then even and differs by one; its potential drops from three to either two or zero, paying for the change.

Above R, the lattice has spacing two. Translation by four preserves ties-to-even rounding, so a gap divisible by four is unchanged. An odd gap becomes even and changes by one, again paid for by the drop in potential. A gap congruent to two modulo four can change only at a tie. When it does, its change has magnitude two and its new value is divisible by four, so the potential decreases by exactly two.

For a straddling pair, the rounding errors have magnitudes at most $1/2$ and 1. Their difference changes the gap by at most $3/2$. Both the old and new gaps are integers, so $|h - g| \le 1$. The potential can increase by at most three, giving a total charge of at most four. An endpoint equal to R can be included in the uniform-lattice case containing the other endpoint; the two lattice descriptions agree at R. □

Theorem 5.2 (Sparse bounded variation)**.** *After m additions under the preceding corridor promise, let $d_i = g_i - 4$. Then*

$$\#\{i : d_i \neq 0\} \le m, \qquad \sum_{i=0}^{L-2} |d_i| \le 4m. \tag{5.3}$$

Proof. At any step, monotonicity permits at most one adjacent pair of exact endpoints to strictly straddle R. Sum the charge from Lemma 5.1 over all gaps. The sum of absolute gap changes plus the change in total potential is at most four. Summing over m steps telescopes. The initial potential is zero and the final potential is nonnegative, so the total variation over all gap histories is at most $4m$. The final distance from the initial gap vector is no larger than this total variation.

For the support bound, an index with zero deviation has gap four. It can first acquire a nonzero deviation only at a crossing. At most one new index is introduced per step, proving the other inequality. □

The asymptotic description bound does not require the numerical constant four to be optimal.

5.2. **Encoding and tightness.**

Theorem 5.3 (Tight corridor description complexity)**.** *For the grid X_* in* (4.3)*, every $p \geq 6$, and every $m \geq 2$,*

$$B(\mathcal{A}(p, m, X_*), X_*) = \Theta\left(m \log_2 \left(1 + \frac{2^p}{m}\right)\right). \tag{5.4}$$

The same order holds for protocols with any fixed pointwise error $0 \leq \varepsilon < 1/2$.

Deterministic upper bound. Set $L = |X_*| = N_*$. The first output has fewer than $2R$ possible integer values and can be encoded using $p + 1$ bits. It and the deviations d_i determine all remaining outputs. Put $n = L - 1$. There are $\binom{n+4m}{4m}$ nonnegative integer vectors of length n with sum at most $4m$. At most $\min\{n, 4m\}$ nonzero coordinates require sign bits. A fixed-width index into these descriptions therefore uses at most

$$p + 1 + \left\lceil \log_2 \binom{L - 1 + 4m}{4m} \right\rceil + \min\{L - 1, 4m\} \tag{5.5}$$

bits.

For $m \leq R$ and $L = \Theta(R)$, standard binomial bounds give $O(m \log_2(1 + R/m))$ bits; the p term is absorbed when $m \geq 2$. For $m > R$, monotonicity gives another code. Encode the nondecreasing integer output ranks by a stars-and-bars bitvector of length $3R/2 + L - 1$. This uses $O(R)$ bits, while $m \log_2(1 + R/m) = \Theta(R)$ in this regime. The public parameters determine which code is used, so this choice requires no block-specific information. □

Deterministic lower bound. Apply Theorem 4.1 on X_* with

$$k = \min\{\lfloor m/2 \rfloor, \lfloor N_*/2 \rfloor\}.$$

For $m \leq N_*$, we have $k \geq m/3$, and $\binom{N_*}{k} \geq (N_*/k)^k$ yields $\Omega(m \log_2(1 + N_*/m))$ bits. For $m \geq N_*$, a central binomial coefficient has logarithm $\Omega(N_*)$. Since $N_* = \Theta(R)$, these regimes give the lower half of (5.4); adjusting universal constants covers the smallest permitted precisions. Section 10 proves the bounded-error lower bound. The deterministic code already supplies the corresponding upper bound. □

5.3. **Constructing the sparse transfer.** The upper representation can be constructed without evaluating all L inputs. Maintain the first output y_0 and the sorted nonzero pairs (j, d_j), together with their prefix sums. Then

$$y_i = y_0 + 4i + \sum_{j<i} d_j. \tag{5.6}$$

When the support has size q, a query takes $O(\log(q + 2))$ comparisons and arithmetic operations. One operand a is processed as follows.

(1) Evaluate the first and last exact sums and their rounded outputs. Reject if either extreme violates the corridor promise.
(2) Find by binary search the first index i for which $y_i + a \geq R$, taking $i = L$ if no such index exists.
(3) Merge the sorted deviation indices with the candidate $i - 1$ when $0 < i < L$, removing a duplicate candidate if necessary.
(4) For each candidate j, compute

$$h = \mathrm{RN}(y_{j+1} + a) - \mathrm{RN}(y_j + a).$$

Retain the new deviation $h - 4$ precisely when it is nonzero.
(5) Replace y_0 by $\mathrm{RN}(y_0 + a)$ and rebuild the prefix sums.

By Lemma 5.1, only the candidate gaps can change. Monotonicity makes the extreme checks sufficient for all exact and rounded trajectories. The update is therefore exact and detects a failed promise without enumerating X. After i operands its support has size at most i. A deterministic sorted-list implementation uses

$$O\Big(m^2 \log(m+2) + m \log L \log(m+2)\Big) \tag{5.7}$$

arithmetic, comparison, and rounding operations for construction. This count excludes serialization and arbitrary-precision bit costs. A sorted merge suffices, so the bound does not depend on average-case hash-table behavior.

The reference serializer uses combinatorial subset ranks for deviation positions and cumulative positive magnitudes, followed by sign bits and the first value. It pads to the largest payload over all support sizes at the same public parameters, and uses the monotone bitvector when that budget is smaller. For compatibility with retained byte artifacts, the tested serializer uses a conservative $5m$ magnitude budget while separately checking the stronger $4m$ invariant. The representation is an evaluator on the specified grid. Closure under arbitrary composition does not follow: one block's outputs can leave the next block's query grid or its residue class.

6. A fixed number of adjacent binades

The two-binade restriction can be relaxed while retaining the original query set. Let $K \geq 2$ and, for this section, assume a normal exponent range containing 0 through $p + K - 1$. Define

$$C_K = [R/2, 2^{K-1}R), \qquad H = 2^{K-1}, \qquad Q = 2H = 2^K. \tag{6.1}$$

The spacings in these K binades are $1, 2, \ldots, H$. Let $\mathcal{A}_K(p, m, X)$ impose the same exact-sum and rounded-state promise as $\mathcal{A}(p, m, X)$, with C_K replacing C. In particular, $\mathcal{A}_2 = \mathcal{A}$. The exponent assumption also makes the upper boundary representable; valid trajectories must still remain strictly below it.

6.1. **A potential for nested binary lattices.** For every nonnegative integer gap, set

$$\psi_Q(g) = Q - \gcd(Q, g), \qquad \gcd(Q, 0) = Q. \tag{6.2}$$

For $Q = 4$, this is precisely (5.1).

Lemma 6.1 (Uniform-lattice charge)**.** *Suppose two exact endpoints at integer distance g are rounded to the same uniform lattice of spacing h, where h is a power of two and $2h$ divides Q. If their rounded distance is g', then*

$$|g' - g| + \psi_Q(g') - \psi_Q(g) \le 0.$$

In particular, a gap divisible by Q is preserved.

Proof. If $2h$ divides g, translation by g preserves both the lattice and the tie parity, so $g' = g$. If h divides g but $2h$ does not, non-ties also preserve the gap. At a tie, $g' = g \pm h$ is divisible by $2h$. The change has magnitude h and the potential decreases by at least h.

Otherwise put $d = \gcd(Q, g) < h$. The new gap is either $h\lfloor g/h \rfloor$ or $h\lceil g/h \rceil$. The nonzero remainder of g modulo h is a multiple of d, so $|g' - g| \le h - d$. Since g' is a multiple of h, its greatest common divisor with Q is at least h. The potential decrease is therefore at least $h - d$, as required. □

Lemma 6.2 (Charge per crossed boundary)**.** *Under the C_K promise, let c be the number of interior binade boundaries strictly between the two exact endpoints of a rounded addition. Then*

$$|g' - g| + \psi_Q(g') - \psi_Q(g) \le Qc. \tag{6.3}$$

Proof. For $c = 0$, Lemma 6.1 applies. An endpoint exactly on a boundary is consistent with the lattice on either side containing the other endpoint: the boundary value is representable with even significand.

Every rounding error has magnitude at most $H/2$. Thus $|g' - g| \le H = Q/2$, and the potential can increase by at most $Q - 1$. The total charge is less than $3Q/2$, which proves (6.3) when $c \ge 2$.

Suppose $c = 1$. If Q does not divide g, then $\gcd(Q, g) \le Q/2$ and the potential increase is at most $Q/2 - 1$. The charge is consequently less than Q. If Q divides g, the endpoints lie on adjacent rounding lattices of spacings h and $2h$. Since $2h \le H = Q/2$, the power of two $4h$ divides Q. Translation by g therefore preserves ties-to-even on both lattices. Writing the lower exact endpoint as u, we obtain

$$g' - g = \mathrm{RN}_{2h}(u) - \mathrm{RN}_h(u),$$

where the subscripts denote uniform-lattice rounding. These two rounded values differ by 0 or $\pm h$. In the nonzero case, $\gcd(Q, g') = h$, so the charge is exactly $h + Q - h = Q$. Otherwise it is zero. □

Theorem 6.3 (Variation across finitely many binades)**.** *Start with a representable arithmetic progression of spacing Q inside C_K. Write $g_i(t)$ for its adjacent output gaps after t additions satisfying the C_K promise. Then*

$$\#\{i : g_i(m) \neq Q\} \leq (K-1)m, \tag{6.4}$$

$$\sum_{t=1}^{m} \sum_{i} |g_i(t) - g_i(t-1)| \leq Q(K-1)m, \tag{6.5}$$

$$\sum_{i} |g_i(m) - Q| \leq Q(K-1)m. \tag{6.6}$$

Proof. At each step the exact endpoints are nondecreasing. Each interior boundary is therefore strictly straddled by at most one adjacent pair, so the sum of their crossing counts is at most $K-1$. Sum Lemma 6.2 over all pairs and steps. The initial potential is zero and the final potential is nonnegative, proving (6.5). The triangle inequality gives (6.6). A gap equal to Q cannot first change without a crossing, by Lemma 6.1; at most $K-1$ new deviation indices can be introduced at a step. This proves (6.4). Zero gaps and coalesced trajectories satisfy the same argument. □

6.2. **The original query grid and a matching bound.** Changing the grid to spacing Q would change the information problem. Instead, partition the original X_* into its nonempty residue classes modulo Q. There are at most $Q/4 = 2^{K-2}$ such classes, each an arithmetic progression of spacing Q.

Theorem 6.4 (Tight description complexity for fixed K)**.** *For every fixed $K \geq 2$, uniformly over $p \geq 6$ and $m \geq 2$,*

$$B(\mathcal{A}_K(p, m, X_*), X_*) = \Theta_K\left(m \log_2\left(1 + \frac{2^p}{m}\right)\right). \tag{6.7}$$

The same order holds for every fixed pointwise error probability $0 \leq \varepsilon < 1/2$, with constants allowed to depend on K and ε.

Proof. Let class j contain L_j queries and put $n_j = L_j - 1$ and $V = Q(K-1)m$. Its first output uses at most $p + K - 1$ bits. By Theorem 6.3, a magnitude vector for its gap deviations has sum at most V. The stars-and-bars argument used in (5.5) therefore gives a fixed-width description of this class using at most

$$p + K - 1 + \left\lceil \log_2 \binom{n_j + V}{V} \right\rceil + \min\{n_j, V\} \tag{6.8}$$

bits. Decode magnitudes first, then read signs in index order, with any remaining sign positions padded. The formula also applies to a singleton class, for which $n_j = 0$.

For fixed K there are a constant number of classes and $V = O_K(m)$. When $m \leq R$, summing (6.8) and using standard binomial bounds gives $O_K(m \log_2(1 + R/m))$ bits. The first-output terms are absorbed because $m \log_2(1 + R/m) \geq \log_2(1 + R) > p$ for $m \geq 1$. For $m > R$, there are exactly $KR/2$ representable states in C_K. Monotone output-rank coding on the

original grid gives a fallback of $O_K(R)$ bits, which has the required order in this regime. The choice of code depends only on public parameters.

Finally, $\mathcal{A}_2(p, m, X_*) \subseteq \mathcal{A}_K(p, m, X_*)$. The threshold lower bound on precisely this X_*, and its bounded-error version in Section 10, therefore apply unchanged. The exact upper code also supplies a bounded-error upper bound. □

The sparse construction extends as well. Maintain one anchor and a sparse gap map for each residue class. At each addition, binary-search each of the $K-1$ boundaries, and update the old deviation indices together with the resulting crossing candidates. Lemma 6.1 proves that no other index can change. For fixed K, a sorted-list implementation has the same asymptotic construction bound as (5.7), with constants depending on K, and still avoids full-grid enumeration. The artifact includes this evaluator; it does not include a new serialized fixed-width codec for arbitrary K.

The fixed-K qualification is essential to the stated estimate. This proof uses up to 2^{K-2} residue classes and a variation budget proportional to $2^K(K-1)m$ per class. It does not give an exponent-range-independent constant when K grows. Theorem 6.4 is an extension of the same gap-potential argument, not a claim of unrestricted exact scan closure.

Some dependence on the exponent range is necessary even independently of this proof. Fix $p = 6$, $m = 2$, so that $X_* = \{40, 44\}$. The zero block and the $K-1$ blocks $(2^{6+j}, 0)$, $0 \le j \le K-2$, satisfy the C_K promise. For each nonzero block put $a = 2^{6+j} \ge 64$. Its exact first sums lie in $[a, a+44]$, strictly below $2a - a/64$, the midpoint between the upper boundary and the preceding representable value. Thus its rounded outputs remain in the binade $[a, 2a)$. These K blocks have distinct outputs already at the query 40, giving the additional lower bound $\lceil \log_2 K \rceil$. The expression $m \log_2(1+2^p/m)$ is constant in this example, so an upper bound with a constant independent of K cannot hold as K tends to infinity.

7. Rounded-state corridors and staircase evaluators

The exact-sum restriction can be removed without changing the preceding information bounds. A different representation also avoids the exponential number of residue classes in the earlier constructor. Removing the exact-sum restriction enlarges the promised class; the new representation controls the number of records needed for exact evaluation when K grows. It does not give a matching information bound uniform in K.

7.1. **A strictly weaker trajectory promise.** Let $\mathcal{A}_K^{\mathrm{rd}}(p, m, X)$ require only that every rounded state belongs to C_K. Retain the format assumptions of Section 6, and put $L = R/2$, $U = 2^{K-1}R$ and $H = 2^{K-1}$ in the following lemma.

Lemma 7.1 (The permitted rounding cells)**.** *For real z,*

$$\mathrm{RN}(z) \in [L, U) \quad \Longleftrightarrow \quad L - \tfrac{1}{4} \le z < U - H/2.$$

On $[L-1/4, R]$, floating-point rounding agrees with nearest-even rounding to the uniform integer lattice.

Proof. The predecessor of L is $L-1/2$, and their midpoint rounds to L. The predecessor of U is $U-H$, and their midpoint rounds to U, which is excluded. Both endpoints have even significands. Monotonicity gives the asserted preimage. The existing normal exponent range includes these neighbours. Every point of the additional fringe $[L-1/4, L)$ rounds to L under both floating-point and uniform integer rounding. Above L and up to R, the original unit-lattice description applies. □

Corollary 7.2 (Rounded-state description complexity)**.** *The gap-variation and support bounds in Theorems 5.2 and 6.3 hold unchanged with only the rounded-state promise. For every fixed $K \geq 2$,*

$$B(\mathcal{A}_K^{\mathrm{rd}}(p, m, X_*), X_*) = \Theta_K\left(m \log_2\left(1 + \frac{2^p}{m}\right)\right) \qquad (p \geq 6,\ m \geq 2).$$

The same order holds for fixed pointwise error below one half. Moreover, $\mathcal{A}_K$ is a proper subclass of $\mathcal{A}_K^{\mathrm{rd}}$ on X_* for every permitted precision and every positive block length.

Proof. The previous rounded states remain integers. Thus a common addition still preserves integer gaps before rounding. Lemma 7.1 extends the lowest uniform-lattice region without introducing a new boundary. Its error on the added fringe is at most $1/4$, within the old $1/2$ bound. All local charge inequalities and their telescoping proofs therefore remain valid. The same codes give the upper bound, and the old threshold family already satisfies the stronger promise, giving both lower bounds.

For strict inclusion use the representable operand $-33/4$ and pad with zeros. At the smallest query $L+8$, its exact sum is $L-1/4$, while its rounded output is L. At every query its output is $x-8$, inside C_K. The operand has six significant bits and normalized exponent three, so it is available under the stated assumptions. The exact-sum promise fails. □

7.2. **Two thresholds for uniform compositions.** Write RN_h for nearest-even rounding on the uniform lattice $h\mathbb{Z}$. The next lemma concerns uniform rounders on the entire real line; their application to floating-point arithmetic will be piecewise.

Lemma 7.3 (Two-threshold normal form)**.** *Let $d > 0$ and all $h_j > 0$ be powers of two. Starting from $x_0 + di$, $i \in \mathbb{Z}$, apply any finite sequence of translations followed by RN_{h_j}. Denote the resulting function by $G(i)$ and set*

$$H = \max\{d/2, h_1, \ldots, h_t\}, \qquad P = 2H/d.$$

Then P is a positive integer. There exist $c = G(0)$ and integers $1 \leq t_1 \leq t_2 \leq P$ such that, writing $i = qP + r$ with $0 \leq r < P$,

$$G(i) = c + 2Hq + H\left(\chi_{r \geq t_1} + \chi_{r \geq t_2}\right), \tag{7.1}$$

where χ denotes an indicator. The thresholds may coincide or equal P.

Proof. Every uniform rounder commutes with translation by $2H$: this displacement is a multiple of twice its spacing and preserves tie parity. Hence $G(i+P) = G(i) + 2H$. The function is nondecreasing.

If an operation has spacing H, immediately after its first occurrence all outputs lie in $H\mathbb{Z}$. A later spacing H has the same property. For a smaller spacing $h \le H/2$, translation by any multiple of H preserves rounding, since $2h$ divides H. Such an operation preserves the property that all output differences are multiples of H, although it can change their common residue. The final outputs therefore lie in the coset $c + H\mathbb{Z}$. If no operation has spacing H, then $H = d/2$; every step preserves the initial spacing d, so $G(i) = c + di$ and the claim follows with $P = t_1 = t_2 = 1$.

For $0 \le i \le P$, monotonicity and equivariance give $c \le G(i) \le c + 2H$. The coset property leaves only the levels $c, c+H, c+2H$. Define $t_j = \min\{i \ge 1 : G(i) \ge c + jH\}$ for $j = 1, 2$, allowing the middle level to be skipped. Equivariance then proves the formula for every integer index, including negative ones. □

For a restriction to $0 \le i < n$, put $T = \min(P, n)$. Each threshold can be found by binary search among $0, \dots, T-1$, with sentinel T if its level is not reached. If $T < P$, all queries have $q = 0$ and $r < T$; clipping the thresholds at T is therefore harmless. No evaluation outside the permitted interval is needed. Rebasing a subinterval at its first index gives the same statement with a new anchor.

7.3. **A constructor polynomial in the number of binades.**

Theorem 7.4 (Piecewise staircase evaluator)**.** *Let $n = |X_*|$ and let a block of length $m \geq 1$ satisfy the rounded-state C_K promise on X_*. Its transfer has an exact representation by at most*

$$q_m = \min\{n, 1 + (K-1)m\}$$

contiguous index intervals, each carrying one two-threshold record. The representation can be built from the operands without enumerating X_*. A query uses $O(\log(q_m + 1))$ comparisons and a constant number of abstract arithmetic operations. A sorted implementation constructs it using

$$O\Big(Km^2 \log(n+1) + Km \log(n+1) \log(Km+2)\Big) \tag{7.2}$$

abstract arithmetic, rounding, and comparison operations. Its records admit a fixed-width description of $O(q_m(p + \log(K+1)))$ bits.

Proof. Initially the entire input grid is one affine piece, with $d = 4$, $H = 2$ and $P = 1$. Maintain a partition on which each previous operation has used a single uniform lattice; the sequence of lattices may differ between pieces. Each piece consequently has the normal form of Lemma 7.3 for the corresponding uniform composition.

For an operand a, the exact values $F(i) + a$ are nondecreasing. Each interior binade boundary has one lower-bound cut index. Merge these at most $K - 1$ indices with the existing piece endpoints. On each resulting subinterval the current operation uses a fixed spacing h. At a boundary itself either adjacent lattice gives the same rounded value; the lower fringe is covered by Lemma 7.1.

For a new subinterval, retain its parent's maximum spacing and replace it by $H' = \max(H, h)$. Evaluate its new first output and find the two thresholds by the bounded binary searches just described. Every probe uses the old record within its own valid interval, followed by one uniform rounding. The underlying uniform composition supplies the normal-form proof even when its extension outside that interval does not model actual floating-point rounding. Each step adds at most $K - 1$ pieces, proving the count.

Two rounded extreme outputs certify the corridor promise by monotonicity. If either fails, the update is rejected before replacing the old records. Locating a boundary costs $O(\log(n+1)\log(q+1))$ using the current piece search. Updating a known subinterval costs $O(\log(n+1))$ probes. Sorting and merging the cuts is absorbed by the displayed bound. Summing over all prefixes, with $q \leq 1 + (K-1)m$, proves (7.2).

For the bit bound, encode piece endpoints and clipped thresholds in $O(\log(n+1)) = O(p)$ bits each. An anchor is one of $KR/2$ floating-point states. The maximum spacing is a power of two with $O(K)$ possible exponents and requires $O(\log(K+1))$ bits. Store its exponent, not an uncompressed binary integer 2^e. Pad to the public maximum record count. These fields determine all queries and give the claimed size. □

This code is not asserted to be information optimal: transmitting the effective operands can use fewer bits. Its purpose is exact evaluation from a constructed index, with a record bound polynomial in K. The previous residue-class constructor can initialize up to 2^{K-2} classes. Neither abstract operation count measures integer bit costs or GPU time. The theorem constructs from the original operands and does not provide an algorithm for merging two descriptions without them.

8. An evaluator on the entire finite format

The staircase representation also extends beyond positive corridors. This is an evaluation theorem, not a matching information lower bound. For this section fix a binary format with precision $p \geq 2$, normal exponents $e_{\min}, \dots, e_{\max}$, and gradual underflow. Put $E = e_{\max} - e_{\min} + 1 \geq 1$ and $\delta = 2^{e_{\min}-p+1}$. Identify the two signed zeros numerically. Inputs and operands are finite representable values. Outputs may be $-\infty$ or $+\infty$; each is absorbing under subsequent finite additions. NaN inputs, nonfinite operands, exception flags, and zero-sign recovery are outside this contract. Every addition is correctly rounded separately to nearest, with ties to even.

Theorem 8.1 (Whole-format exact index)**.** *There are*

$$N = 2^p(E+1) - 1$$

finite numerical input values. For every block of $m \geq 0$ finite operands, its sequential transfer on all these inputs admits an exact index with at most

$$q_m = \min\{N, 2E + 1 + 2Em\} \tag{8.1}$$

contiguous input-rank pieces. Each finite-output piece uses a two-threshold record; infinite-output pieces are constant. The index can be constructed without an initial table of all N values. Queries use $O(\log(E(m+1)+2))$ abstract operations, including input-rank conversion. A sorted implementation has construction cost

$$O\Big(E + Em^2 \log(N+2) + Em \log(N+2) \log(E(m+1)+2)\Big). \tag{8.2}$$

There is a fixed-width encoding using $O(q_m(p + \log(E+1)))$ bits. These are arithmetic-operation and information bounds, not unit-cost assertions about arbitrary-size integers.

Proof. Scale all values by $1/\delta$. The subnormal step becomes one. Each normal binade has 2^{p-1} values and spacing 2^j for some $0 \leq j < E$. The positive subnormals contribute $2^{p-1} - 1$ more values. Including negatives and one numerical zero gives the displayed N. In increasing order, the finite inputs are the union of E negative binade progressions, one central progression of subnormals and zero, and E positive binade progressions. Thus the identity starts with $2E + 1$ affine pieces. Each contains fewer than 2^p inputs.

Let $h_{\max} = 2^{E-1}$ in the scaled units. The largest finite number is $M = (2^p - 1)h_{\max}$, and the overflow threshold is

$$\tau = M + h_{\max}/2.$$

Nearest-even rounding returns $+\infty$ at $z \geq \tau$ and $-\infty$ at $z \leq -\tau$, including the midpoint ties. Between these tails, rounding agrees with uniform dyadic rounders on $2E - 1$ consecutive regions. The central region includes both subnormals and the lowest normal binades. Its spacing is one. The other region boundaries are

$$\pm 2^{p-1+j}, \qquad 1 \leq j < E.$$

Adjacent uniform descriptions agree at each boundary itself.

Suppose the current transfer is indexed. Addition of a finite operand preserves monotonicity, also on the extended ordered set containing the two infinities. Partition its exact outputs at the $2(E-1)$ interior boundaries and the two overflow thresholds. Each has one preimage cut in the entire ordered input domain. For the negative overflow threshold use the first exact output strictly greater than $-\tau$; for the positive threshold use the first output at least τ. Interior cuts may use the latter convention. These choices include both overflow ties in their absorbing tails. Existing infinite outputs remain infinite.

After merging these at most $2E$ cuts with old piece endpoints, each new piece either has one constant infinite output or uses one uniform rounding lattice. In the latter case its earlier finite history and this operation form a uniform composition on its original dyadic input progression. Lemma 7.3 applies with that progression's spacing d. Rebase at the new piece start, update H, and recover its anchor and two clipped thresholds by binary search within the piece. No history can return from infinity to a finite state, because all operands are finite. Induction gives (8.1).

Input-rank conversion locates one of the initial progressions and then uses an exact offset and division by its dyadic spacing. Both this search and the record search take logarithmically many comparisons. A record uses (7.1) or returns its constant infinity. For construction, each region cut requires $O(\log(N+2))$ index probes, each costing $O(\log(q+1))$. A known new piece requires only $O(\log(N+2))$ constant-cost probes to its parent record. Sorting and merging are absorbed by these bounds. Summing over all steps gives (8.2), with $O(E)$ initialization.

Finally, piece endpoints are ranks in a set of size N and anchors are finite format values or infinities. Each uses $O(p + \log(E+1))$ bits. The original input spacing and the maximum rounding spacing have $O(E)$ possible dyadic exponents in the scaled format; the initial fictitious half-spacing adds only one possibility. Each clipped threshold is at most its piece length, which is less than 2^p since the initial progression boundaries are retained. Encoding these fields, the piece count, and padding to q_m proves the bit bound. □

The finite-domain representation can reach N pieces; the theorem does not guarantee a small index for every length and format. Its bound is polynomial in the number of binades and the block length, without requiring enumeration of all significands. The lower bounds on the corridor families remain consistent with this larger index. Transmitting the operands is still a smaller general description in many regimes. The index trades preprocessing and space for queries that do not replay the block. An efficient merge of two already constructed indices, and any benefit on a concrete parallel workload, remain separate questions.

9. Order information with a public numerical multiset

The threshold family varies operand magnitudes. We now fix the entire multiset and isolate the information carried by operand order.

Theorem 9.1 (Factorial attainability)**.** *Let $k \geq 1$ and $R = 2^p \geq 64k^2$. There is a public multiset of $4k$ distinct finite representable operands with permutations realizing at least $k!$ different transfers on one common set of k^2 incoming markers. Every constructed block has exact real sum zero and satisfies $F(0) = 0$. On the markers, all its exact sums and rounded states lie in $[3R/4, 2R)$. Thus an exact fixed-width description requires at least $\lceil \log_2(k!) \rceil$ bits even when the multiset is public.*

Proof. Index i, j from zero to $k-1$. Set $A = 3R/4$ and define

$$\begin{aligned} X &= \{A + 4\ell : 1 \leq \ell \leq k^2\}, \quad s_i = R/16 + 4i, \\ t_j &= 13R/16 + 4k(j+1), \qquad r_j = R + c_j - t_j, \end{aligned} \tag{9.1}$$

where c_j is one for even j and four for odd j. Given a permutation π of $\{0, \ldots, k-1\}$, concatenate the k slots

$$(+s_{\pi(j)}, +r_j, -r_j, -s_{\pi(j)}), \qquad 0 \leq j < k. \tag{9.2}$$

All operands are integers of magnitude less than R, and hence are representable. The shifts are distinct and satisfy

$$R/16 \leq s_i \leq R/8 - 4.$$

The radii satisfy $R/8 + 1 \leq r_j \leq 3R/16$ and strictly decrease: consecutive t_j differ by $4k$, whereas consecutive c_j differ in magnitude by only three. The two families of positive magnitudes are disjoint and contain no repetitions. Including their negatives gives $4k$ distinct operands, each appearing exactly once in every block. Each slot has exact real sum zero. Starting from zero, its integer partial sums have magnitude below R, so it also returns exactly to zero in the floating-point format.

Slot j has effective threshold

$$\tau_j = t_j - s_{\pi(j)} = A + 4\big(k(j+1) - \pi(j)\big). \tag{9.3}$$

This threshold occupies one of the k markers numbered $kj+1$ through $k(j+1)$. The bands are disjoint and strictly ordered. The thresholds therefore increase with j, and their positions determine π.

For a marker x, let d be the parity of the number of thresholds already crossed. Suppose its state before the slot is $x - d$, as is true initially. Put $y = x + s_{\pi(j)}$. The range assumptions give

$$A + 4 \le x \le 13R/16, \qquad 13R/16 + 4 \le y \le 15R/16 - 4.$$

The first shift is exact and gives $y - d$. Both y and t_j are divisible by four.

If $x < \tau_j$, then $y \le t_j - 4$. The first sum of the inner pair is a positive integer at most R. Both inner operations are exact and preserve $y - d$. If $x \ge \tau_j$, then all j earlier thresholds have been crossed, so $d = j \bmod 2$. For even j, the first inner sum is $R+1+y-t_j$ and rounds down by one. For odd j, it is $R + 3 + y - t_j$ and rounds up by one. Exact subtraction of r_j gives $y - 1$ or y, respectively. The final subtraction of $s_{\pi(j)}$ is also exact, yielding $x - 1$ or x. The largest exact inner sum is at most $9R/8 - 4$, and rounding adds at most one; every exact and rounded intermediate value remains in $[A, 2R)$.

Thus

$$F_\pi(x) = x - \big(\#\{j : \tau_j \le x\} \bmod 2\big). \tag{9.4}$$

As in Theorem 4.1, the first threshold in a symmetric difference distinguishes two transfers. Its band position recovers the assigned shift. Hence all $k!$ permutations in the family give different transfer functions. □

For $m = 4k$ and $R \ge 4m^2$, this gives $\Theta(m \log m)$ bits of additional order information as k grows. The matching upper bound is an index among at most $m!$ orders of the public multiset. This is a worst-case result over numerical multisets in the stated precision regime, not a lower bound for every multiset. Common zero padding preserves the $k!$ witnesses at longer lengths, but does not permit replacing k by an arbitrarily larger m in the asymptotic bound. For $m = 256$ and $k = 64$, the exact deterministic lower bound is 296 bits whenever the precision condition holds. This conditional-multiset bound concerns a different interface from the unrestricted one in Section 4.

10. Pointwise bounded-error summaries

10.1. **Entropy lemma.** Write

$$h_2(\varepsilon) = -\varepsilon \log_2 \varepsilon - (1 - \varepsilon) \log_2(1 - \varepsilon),$$

with $0 \log_2 0 = 0$. The following standard entropy argument separates the information inequality from the arithmetic reductions.

Lemma 10.1 (Conditional bit recovery)**.** *Let $B_1, \dots, B_q$ be independent uniform bits, let U be public randomness independent of them, and let M be a b-bit message. Suppose that, for every t, an estimator given $M, U, B_1, \dots, B_{t-1}$ recovers B_t with error probability at most ε, where $0 \le \varepsilon < 1/2$. Then*

$$b \ge (1 - h_2(\varepsilon))q. \tag{10.1}$$

Proof. Let E_t indicate whether the estimator is wrong. Given the estimator's inputs, E_t determines B_t. The binary entropy bound therefore gives

$$H(B_t \mid M, U, B_1, \dots, B_{t-1}) \le h_2(\varepsilon).$$

The chain rule yields

$$H(B_1, \dots, B_q \mid M, U) \le q h_2(\varepsilon).$$

Independence from U gives $H(B_1, \dots, B_q \mid U) = q$. Consequently,

$$I(B_1, \dots, B_q; M \mid U) \ge q(1 - h_2(\varepsilon)).$$

The same mutual information is at most $H(M \mid U) \le b$. □

In the applications below, the query recovering a bit depends only on earlier bits and public parameters, not on M or U. The pointwise guarantee for each fixed block-query pair thus bounds the averaged decoding error in Lemma 10.1. No simultaneous guarantee over all queries of one random summary is needed.

10.2. **Threshold choices.** Let N denote the size of either threshold grid, and choose

$$1 \le k \le \min\{\lfloor m/2 \rfloor, \lfloor N/2 \rfloor\}, \qquad q = \lfloor \log_2(N/k) \rfloor \ge 1.$$

Partition the first $k2^q$ markers into k consecutive groups of 2^q markers. In each group choose one threshold whose rank is an independent uniform q-bit number. Theorem 4.1 constructs a block for every such choice.

Order the bits first by group and then, within a rank, from most to least significant. Given the higher bits of a rank, its possible values form a dyadic interval. Query the marker at the last rank in the lower half of that interval. After removing the known parity of earlier groups, the transfer output says whether the selected threshold is at most this marker, and hence determines the next bit. Lemma 10.1 gives

$$b \ge (1 - h_2(\varepsilon)) k \lfloor \log_2(N/k) \rfloor. \tag{10.2}$$

For the corridor grid, take $k = \min\{\lfloor m/2 \rfloor, \lfloor N_*/2 \rfloor\}$. This yields $\Omega(m \log_2(1 + R/m))$ before saturation and $\Omega(R)$ afterward, completing the bounded-error part of Theorem 5.3. Its constants may depend on the fixed error ε. Together with the operand-replay upper bound, the same argument gives $\Theta(mp)$ in the regime of Corollary 4.2.

For $m \ge 2N$, a direct construction covers saturation. Encode an arbitrary vector of N bits by selecting a threshold wherever consecutive bits differ, taking the bit before the first marker to be zero. At most N threshold pairs are needed. Common zero padding fixes the length, and the output at each marker reveals its bit. Lemma 10.1 gives $b \ge (1 - h_2(\varepsilon))N$. In the deterministic case, this subfamily requires exactly N bits.

10.3. **Permutation ranks.** In Theorem 9.1, order the remaining shifts at slot j by their effective threshold. There are $k-j$ choices. Set $q_j = \lfloor \log_2(k-j) \rfloor$, retain the first 2^{q_j} ranks, and choose a rank using independent uniform q_j bits. Earlier choices determine the remaining ordered set, so every chosen rank has an available shift. Distinct rank sequences produce distinct permutations.

Given the earlier slots and the higher bits of the current rank, query the marker at the last remaining threshold in the lower half of the current dyadic rank interval. It is an allowed marker in slot j's band. All previous slots have crossed, no later slot has crossed, and the current output parity determines the next rank bit. Applying Lemma 10.1 to the concatenated bits gives

$$\begin{aligned} b &\geq (1 - h_2(\varepsilon)) \sum_{u=1}^{k} \lfloor \log_2 u \rfloor \\ &\geq (1 - h_2(\varepsilon)) \big(\log_2(k!) - k \big). \end{aligned} \tag{10.3}$$

Thus the $\Theta(m \log m)$ order-information bound also holds for every fixed pointwise error below one half, in the same precision regime. The second expression in (10.3) may be uninformative for small k; the exact sum is always nonnegative.

All these bit-recovery arguments also apply when, with probability at least $1 - \varepsilon$, the numerical answer differs from the correct result by strictly less than $1/2$. The two possible answers at a query differ by one, so their permitted answer intervals are disjoint. An absolute error allowance of one destroys these particular witnesses: returning x then succeeds without any block-dependent message.

11. Validation and reproducibility

11.1. **Baselines and evaluation criteria.** We compare description sizes. We use three exact baselines: transmitting the m operand words and replaying the block after x arrives; storing every query result; and encoding a nondecreasing output sequence by a monotone bitvector. For the constructed families, subset and permutation ranks give the corresponding information upper bounds. These comparisons concern all representations obeying the access contract, rather than the properties of one implementation.

The reference implementation uses exact rational arithmetic to construct sparse gap maps and fixed-width byte messages. A full-table oracle checks the same ordered specification on small domains, and the monotone fallback is implemented. The serialized size is distinguished from Python object memory. Native CPU and GPU runs test arithmetic correctness, not throughput. The reference is not claimed to outperform a native sequential fold or to provide a GPU optimization.

The artifact also retains an exploratory arithmetic implementation of the Kapre-DeHon iterative scheme [12]. It does not reproduce their FPGA implementation or benchmark a current fastest kernel. Its correction counts

TABLE 1. Completed validation. Counts refer to executed checks, not planned experiments.

Check	Coverage	Outcome
Exact threshold enumeration	16,452 subsets; 229,768 marker folds	Expected cardinalities
Exact permutation enumeration	873 permutations; 29,367 marker folds	Exactly $k!$ signatures
Factoradic query reduction	1,367 codewords; 12,746 bit queries	Every bit recovered
Corridor block enumeration	11,652 admissible blocks; 60,036 steps	Conservative invariant holds
Original local symbolic checks	196 counterexample queries	All UNSAT
Independent symbolic checks	7 local obligations, including the weighted $4m$ bound	All UNSAT
Independent finite-format arithmetic	59,528 additions at $p = 2, 3, 4$, including exceptional numerical cases	Oracle checks pass
Independent corridor exploration	142,340 reachable-state transitions	Support and $4m$ bounds hold
Native CPU arithmetic	120,925 queries in binary16, binary32, and binary64	No output mismatches
Actual Apple M5 Pro GPU arithmetic	27,713 queries in binary32	No output mismatches
Reference constructor	144 blocks; 2,280 prefixes; 107,920 full-table comparisons	Pass with $4m$ assertion
Independent reference cross-check	84 blocks; 18,240 queries; 3,444 serialization round trips	Exact agreement

refer to a different data-access contract and are not an optimality baseline for the standalone summary.

11.2. **Completed checks.** Table 1 lists the completed checks. The saved hardware outputs are kept separate from subsequent comparisons against those outputs; the latter are not counted as additional device runs.

The seven independent symbolic obligations cover monotone rounding, residue-potential decrease away from the boundary, a crossing's gap change of at most one, preservation of a gap of four away from a crossing, both weighted local inequalities, and the threshold-pair transition. They are machine checks of local lemmas, not a proof-assistant formalization of the complete theorems. Earlier unsplit symbolic searches that timed out remain recorded as unsuccessful attempts.

The independent arithmetic oracle includes signed zeros, gradual underflow, and overflow to test numerical cases beyond the witness trajectories. Finite

enumeration does not establish a theorem at arbitrary precision; the general arguments are in Sections 4–10. Likewise, the reachable-state count records explored transitions and does not claim to exhaust all real operand sequences.

The CPU records include strict rounding settings and scalar floating-add instructions. The GPU records include the device, compilation options, input buffers, and actual output bits. They contain 7,560 threshold queries and 20,153 permutation queries, totaling 27,713. The later sparse-constructor comparison with the 20,153 permutation outputs reuses those saved bits. Software arithmetic models are recorded separately from these device runs.

The fixed-K extension was checked separately using exact arithmetic. A uniform-lattice enumeration covered 347,376 cases, and 1,339,284 local endpoint pairs checked the boundary charge. Two independently implemented rounders agreed on 8,608 individual exact inputs. A further 600 valid blocks gave 20,520 checked prefixes and 430,920 full-table query comparisons, with up to six binades. An independently written rational oracle checked 11,169,226 uniform-lattice cases, 2,509,836 boundary pairs, and 6,144 block prefixes with 122,880 queries; its finite encoding test made 11,181 round trips. These counts describe executed finite checks, not a formal proof of all parameters.

The new sparse evaluator passed 240 block tests, comprising 8,208 prefixes and 103,968 full-table comparisons, and rejected three corridor violations without changing its state. Additional tests with nonbinary rational addends checked the rounding implementation but are not claimed as floating-point operand blocks. For $K = 3$ and $p = 24, 53$, the representable operand $3R/2 + 2$ produces alternating gaps eight and zero on X_*. The evaluator represents this map by two spacing-eight classes with no deviation entries. At each precision, 518 selected queries agreed with direct exact rounding. The domain sizes are the same as in Table 3; these are evaluator tests, not new serialized messages or hardware measurements.

11.3. **Staircase indices and the complete format.** The rounded-state relaxation was checked on 1,495,456 local endpoint pairs, including 2,776 lower-fringe pairs, and 560 blocks with 19,152 prefixes and 402,192 full-grid comparisons. Of those prefixes, 562 actually violate the old exact-sum promise while satisfying the new one. The proof, rather than this finite enumeration, establishes the relaxation at every permitted precision.

An independently written normal-form checker tested 87,165 short uniform compositions and 160 longer chains, making 904,342 query comparisons. Its separate corridor piece updater made 402,192 full-grid comparisons. The main staircase implementation passed 128 blocks, 4,096 prefixes, and 339,968 full-grid comparisons; a separately written cross-check added 104,580 comparisons. These tests cover coincident thresholds, clipped periods, negative index rebasing in the uniform lemma, and preservation of a coset after a finer rounding.

TABLE 2. Whole-format indices for an eight-operand block. Counts are records, not bytes or speed measurements. Signed zeros count as one input.

Quantity	binary32	binary64
Normal binades E	254	2,046
Finite numerical inputs N	4,278,190,079	18,437,736,874,454,810,623
Constructed records	1,256	7,992
Sampled native-CPU queries	6,165	16,891
Numerical mismatches	0	0

For the whole-format reference, an enumerated-neighbour oracle checked 30,018 exact sums. All finite inputs were then checked after every prefix of 984 exhaustively generated short blocks and 400 additional blocks: 10,404 prefixes and 912,216 query comparisons in total. Tests cover negative binade boundaries, gradual underflow, both overflow midpoint ties, and the persistence of infinity after a later finite operand. A wrong lattice-side choice at a negative boundary was detected in the prototype and corrected before these final runs.

Table 2 gives two additional whole-format indices. The same eight-operand pattern in each format was

$$(\delta, 1, -1, 2^p, 1, -2^p, M, -M),$$

where δ is its minimum positive subnormal and M its largest finite value. These indices cover the entire finite numerical input domain by Theorem 8.1; their construction did not enumerate that domain. Sampled inputs include piece endpoints and random input ranks. Their outputs were compared against actual CPU additions using NumPy 2.3.5 on arm64 macOS, with an explicit binary32 or binary64 dtype at every addition. Subnormal preservation was checked. The input, operand, and output bits are retained. The 23,056 sampled queries used 184,448 separate CPU additions and produced no numerical mismatches. These are arithmetic correctness checks, not timings or new GPU runs.

The corridor constructor was also tested at $p = 24, 53$ and $K = 3, 20, 64$. Each of six examples used 33 records on grids of 1,048,574 or 562,949,953,421,310 queries, with 322 sampled queries checked against exact arithmetic. The record count is not a comparison against the smallest possible encoding; some blocks have further compressible outputs.

11.4. **Serialized examples on large domains.** For two length-256 threshold blocks with 128 selected thresholds, the sparse constructor was run on the corridor grid at precisions 24 and 53. The representations were constructed without traversing the full query domain. Table 3 gives their sizes and construction counts.

TABLE 3. Serialized corridor examples with $m = 256$. The public budgets use the conservative $5m$ serializer.

Quantity	$p = 24$	$p = 53$
Incoming states	1,048,574	562,949,953,421,310
Actual payload bits	2,602	6,343
Public worst-case bit budget	4,645	12,099
Serialized bytes	581	1,513
Rounding evaluations	33,536	33,536

The payload depends on the block, but every message is padded to the public worst-case budget. The byte counts include final byte padding and exclude shared format, grid, and length metadata. Both examples use the conservative $5m$ magnitude budget retained by the serializer; the $4m$ theorem permits a tighter information bound. At each precision, 642 sampled queries agree with direct exact folds. Full-table tests on small domains, rather than those samples alone, check the general implementation path. The table reports sizes and operation counts, not running time or peak memory.

12. Limitations and open questions

12.1. **The corridor and rounding hypotheses.** The original spacing-four invariant fails outside two binades. At $p = 7$, let $R = 128$ and apply the exactly representable operand 512 to the six markers

$$72, 76, 80, 84, 88, 92.$$

The rounded outputs are

$$584, 592, 592, 592, 600, 608.$$

Their gap deviations from four are $4, -4, -4, 4, 4$, with absolute sum 20 and support five. Both exceed the one-step bounds in Theorem 5.2. The failure already occurs in three binades: at $p = 8$, the representable operand 386 maps $X_* = \{136, 140, \dots, 188\}$ to gaps alternating eight and zero. All 13 gaps deviate from four and their total deviation is 52 after one addition. Partitioning X_* into its two residue classes modulo eight removes this apparent obstruction to Theorem 6.4; it does not restore the original spacing-four support estimate.

The boundary count in Lemma 6.2 is also necessary for that particular charge bound. At $p = 6$, $K = 4$, $Q = 16$, exact endpoints 51 and 291 round to 51 and 288. The gap changes from 240 to 237, giving charge $3 + 15 - 0 = 18 > Q$. The endpoints strictly cross three boundaries, so the proved bound Qc still holds.

The tie rule also matters. At $p = 4$, the block $(+9, -9)$ maps $x = 8$ to seven under ties-to-even and to nine under ties away from zero. The threshold proof cannot simply substitute another rounding mode. Moreover,

$(+9, -9)$ and $(+5, -5)$ both have real sum zero and return zero from zero, but map eight to seven and eight, respectively, under ties-to-even. Neither the correctly rounded real sum nor the zero-start fold determines the required transfer.

The exact-sum condition is strictly stronger than the rounded-state condition. At $p = 7$, the operand $-33/4$ sends the smallest query 72 to the exact sum 63.75 and the rounded state 64. It violates the former promise but satisfies the latter on the entire query grid. Corollary 7.2 proves that the same information and variation bounds still apply. The original sparse implementation retains its stricter admission check; the new staircase constructor accepts the relaxed class.

The zero-start identity in Theorem 9.1 applies to its constructed orders, not every permutation of the public multiset. For $p = 11$ and $k = 5$, list all positive operands as

$$128, 132, 136, 140, 144, 365, 348, 325, 308, 285$$

and then append their negatives in the same order. The block returns one from zero. The slot-form orders used in the proof return zero.

12.2. **The access contract and composition.** Interaction changes the information problem. A decoder can send x to the block owner and receive $F_a(x)$ using two floating-point words. If x is already known during encoding, one output word suffices. These alternatives lie outside the unknown-entry one-way contract. The lower bounds also permit algorithms that revisit the original operands, use distributed storage charged elsewhere, or perform iterative speculative correction.

The corridor sparse representation is an evaluator on its specified grid. That grid need not contain the outputs of another block, so an exact associative scan format would require an additional closure argument. The implementation also checks a narrower starting domain than the abstract spacing-four lemma; the grid X_* satisfies both. Its construction bound counts abstract arithmetic operations and does not establish a bit-complexity or GPU-work bound.

12.3. **Unresolved questions.** The matching fixed-K information result leaves open a tight joint dependence on length, precision, and the number of binades when K grows. The new staircase index has polynomial record dependence, but is generally larger than the operand list and does not close that information gap. The whole-format theorem removes positivity for evaluation; it identifies signed zeros numerically and omits NaNs, nonfinite operands, and exception flags. An efficient algorithm for merging two constructed indices remains open. A separate systems question is whether constructing and applying summaries pays for its cost on a concrete workload. Answering it requires optimized implementations, a specified memory-access contract, and comparable timing measurements. Our results do not settle either question.

13. Conclusion

Ordered rounded additions realize many distinct functions of the incoming accumulator, even when the real sums and zero-start folds agree. Threshold and permutation constructions turn this distinction into description lower bounds. On a common two-binade query domain, a residue potential bounds total gap variation by $4m$, yielding a sparse evaluator and a matching $\Theta(m \log_2(1 + 2^p/m))$ bound. A greatest-common-divisor potential and a partition of the same query grid extend that order to every fixed number of adjacent positive binades, and the same bounds hold under the weaker rounded-state promise. A different two-threshold normal form gives an exact index with $O(mK)$ corridor pieces and $O(E(m + 1))$ pieces over the entire finite format, including negative values, subnormals, and absorbing infinities. These evaluation indices need not minimize description size. Standard entropy arguments extend the lower bounds to pointwise bounded error. None of these results is a running-time lower bound for unrestricted parallel algorithms or a measured speedup.

Appendix A. Reproduction

The accompanying research artifact, version 1.0.0, is available at https://doi.org/10.5281/zenodo.24044237. Its `README.txt` describes the package, `REPRODUCE.txt` gives the dependencies and commands, and `EVIDENCE_MAP.json` maps the paper's validation results to code and data. The source code is in `research/experiments/`, `research/adversarial/`, and `research/extensions/`; recorded outputs and environment information are in `research/results/` and the relevant extension directories.

The following commands run from `research/`. The symbolic checks and the independent reference cross-check require the Z3 Python package. The boundary-attack, corridor-summary, and retained-record verification commands use the standard library.

```
python3 -B adversarial/root/boundary_attacks.py
python3 -B adversarial/math/independent_audit.py
python3 -B adversarial/math/reference_crosscheck.py
python3 -B experiments/check_corridor_summary.py
python3 -B experiments/verify_artifact.py -public-bundle
```

The last command checks retained code and data without rerunning the GPU experiments. Device build and execution instructions are in `REPRODUCE.txt` at the package root. Saved outputs support checks on machines without the original GPU but do not replace an independent device run. The independent reference cross-check used seed 89623471; the other seeds are recorded in their scripts and result files. The fixed-K extension and its separate checks are in `research/extensions/multibinade_2026_11_07/`. Its two main commands are `python3 -B check_multibinade.py` and `python3 -B check_sparse.py`, run from that directory. The independent oracle is `independent_review/check_independent.py`.

The rounded-state, staircase, and full-format extensions are in `research/extensions/strengthening_2026_11_07/`. Their principal commands, run from that directory, are

```
python3 -B check_staircase.py
python3 -B independent_math/check_rounded_corridor.py
python3 -B independent_math/check_uniform_pieces.py
python3 -B independent_math/check_full_format.py
python3 -B check_full_native.py
```

The last command requires NumPy and executes new CPU arithmetic; it does not use a GPU.

The historical comparison in Section 2 used the complete Reiser-Knuth original, its corrections, the corrected 2010 chapter and addendum, and Diamond's article. Knuth's exercise 4.2.2–22 and its answer were compared between the second edition (pp. 229 and 573) and the third edition (pp. 244 and 616). Both give the fixed-multiplier stabilization result; the later version explicitly excludes $v = 0$ and revises a boundary-case argument. This comparison does not certify an exhaustive priority search.

Use of AI tools

Generative AI assisted with developing and checking mathematical arguments, writing and testing code, comparing the literature, and editing the manuscript; the author takes responsibility for its contents.

References

1. Roberto Bagnara, Abramo Bagnara, Fabio Biselli, Michele Chiari, and Roberta Gori, *Correct approximation of IEEE 754 floating-point arithmetic for program verification*, Constraints **27** (2022), 29–69, https://doi.org/10.1007/s10601-021-09322-9.
2. Julien Brandoit, Arthur Fyon, Thomas Braipson, Tom Clara, Florent De Geeter, Pierre Sacré, Damien Ernst, and Guillaume Drion, *Length-independent state tracking under a parallel scan*, arXiv:2609.32447v1, 2026, https://doi.org/10.48550/arXiv.2609.32447.
3. Sílvia Casacuberta, Michael Shoemate, Salil Vadhan, and Connor Wagaman, *Widespread underestimation of sensitivity in differentially private libraries and how to fix it*, Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS '22), ACM, 2022, Full version: arXiv:2207.10635v2; https://doi.org/10.1145/3548606.3560708.
4. Amit Chakrabarti, Graham Cormode, Ranganath Kondapally, and Andrew McGregor, *Information cost tradeoffs for augmented index and streaming language recognition*, arXiv:1004.3304v1, 2010, https://doi.org/10.48550/arXiv.1004.3304.
5. Harold G. Diamond, *Stability of rounded off inverses under iteration*, Mathematics of Computation **32** (1978), no. 141, 227–232, https://doi.org/10.1090/S0025-5718-1978-0461879-7.
6. Roger B. Eggleton, *The well-rounded linear function*, The Electronic Journal of Combinatorics **8** (2001), no. 2, R6, https://doi.org/10.37236/1605.
7. *Errata*, Information Processing Letters **3**, no. 5, p. 164, 1975, First correction block: Reiser and Knuth (1975); https://doi.org/10.1016/0020-0190(75)90031-9.
8. Luca Foschini, John Hershberger, and Subhash Suri, *On the complexity of time-dependent shortest paths*, Proceedings of the Twenty-Second Annual ACM-SIAM

Symposium on Discrete Algorithms (SODA 2011), SIAM, 2011, https://doi.org/10.1137/1.9781611973082.27, pp. 327–341.
9. Robert Joseph George, Will Adkisson, and Anima Anandkumar, *FloatLib: Verified floating-point arithmetic in Lean*, 2026, arXiv:2609.19352v1 [cs.LO], September 16, 2026; https://arxiv.org/abs/2609.19352v1.
10. David Goldberg, *What every computer scientist should know about floating-point arithmetic*, ACM Computing Surveys **23** (1991), no. 1, 5–48, Oracle reprint; https://docs.oracle.com/cd/E19957-01/806-3568/ncg_goldberg.html.
11. Monika Henzinger, Stefan Neumann, Harald Räcke, and Stefan Schmid, *Dynamic maintenance of monotone dynamic programs and applications*, 40th International Symposium on Theoretical Aspects of Computer Science (STACS 2023), Leibniz International Proceedings in Informatics (LIPIcs), vol. 254, Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2023, https://doi.org/10.4230/LIPIcs.STACS.2023.36, pp. 36:1–36:16.
12. Nachiket Kapre and André DeHon, *Optimistic parallelization of floating-point accumulation*, 18th IEEE Symposium on Computer Arithmetic (ARITH 2007), IEEE, 2007, https://nachiket.github.io/publications/fp-accum_arith-2007.pdf.
13. Donald E. Knuth, *The art of computer programming, volume 2: Seminumerical algorithms*, 2nd ed., Addison-Wesley, Reading, MA, 1981, Exercise 4.2.2–22, p. 229; answer, p. 573; https://www-cs-faculty.stanford.edu/~knuth/taocp.html.
14. ———, *The art of computer programming, volume 2: Seminumerical algorithms*, 3rd ed., Addison-Wesley, Reading, MA, 1997, Exercises 4.2.2–22 and 4.2.2–28, pp. 244–245; solutions, pp. 616–617; https://www-cs-faculty.stanford.edu/~knuth/taocp.html.
15. Richard E. Ladner and Michael J. Fischer, *Parallel prefix computation*, Journal of the ACM **27** (1980), no. 4, 831–838, https://doi.org/10.1145/322217.322232.
16. Jeffrey C. Lagarias, Takumi Murayama, and D. Harry Richman, *Dilated floor functions that commute*, The American Mathematical Monthly **123** (2016), no. 10, 1033–1038, https://doi.org/10.4169/amer.math.monthly.123.10.1033.
17. Wonyeol Lee, Rahul Sharma, and Alex Aiken, *Verifying bit-manipulations of floating-point*, Proceedings of the 37th ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI '16, Association for Computing Machinery, 2016, https://doi.org/10.1145/2908080.2908107, pp. 70–84.
18. Laura Monroe and Vanessa Job, *Computationally inequivalent summations and their parenthetic forms*, arXiv:2005.05387v1, 2020, https://doi.org/10.48550/arXiv.2005.05387.
19. Todd Mytkowicz, Madanlal Musuvathi, and Wolfram Schulte, *Data-parallel finite-state machines*, Proceedings of the Nineteenth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '14), ACM, 2014, https://doi.org/10.1145/2541940.2541988.
20. Franz Nowak, Ryan Cotterell, and Reda Boumasmoud, *An algebraic view of the expressivity of recurrent language models*, arXiv:2606.01765v3, 2026, Accepted at the 43rd International Conference on Machine Learning (ICML 2026); https://doi.org/10.48550/arXiv.2606.01765.
21. ———, *A compositional theory of causally masked transformers*, arXiv:2607.26988v1, 2026, https://doi.org/10.48550/arXiv.2607.26988.
22. Karl Papadantonakis, Nachiket Kapre, Stephanie Chan, and André DeHon, *Pipelining saturated accumulation*, IEEE Transactions on Computers **58** (2009), no. 2, 208–219, https://doi.org/10.1109/TC.2008.110.
23. John F. Reiser and Donald E. Knuth, *Evading the drift in floating-point addition*, Information Processing Letters **3** (1975), no. 3, 84–87, https://doi.org/10.1016/0020-0190(75)90022-8.
24. ———, *Evading the drift in floating-point addition*, Selected Papers on Design of Algorithms, CSLI Lecture Notes, vol. 191, Center for the Study of Language and

Information, Stanford, CA, 2010, Corrected reprint with addendum on p. 348; https://www-cs-faculty.stanford.edu/~knuth/da.html, pp. 341–348.
25. Brett Saiki, Bill Zorn, Cynthia Richey, and Zachary Tatlock, *When double rounding is correct*, 2026, arXiv:2610.09005v2 [cs.PL], October 15, 2026; https://arxiv.org/abs/2610.09005v2.
26. Sebastiano Vigna, *Quasi-succinct indices*, Proceedings of the 6th ACM International Conference on Web Search and Data Mining (WSDM '13), ACM, 2013, https://vigna.di.unimi.it/ftp/papers/QuasiSuccinctIndices.pdf, pp. 83–92.
27. Václav Voráček and Matthias Hein, *Sound randomized smoothing in floating-point arithmetics*, International Conference on Learning Representations (ICLR 2023), 2023, Full version: arXiv:2207.07209v2; https://arxiv.org/abs/2207.07209v2.
28. Peichen Xie, Yanjie Gao, Yang Wang, and Jilong Xue, *Revealing floating-point accumulation orders in software/hardware implementations*, 2025 USENIX Annual Technical Conference (USENIX ATC 25), USENIX Association, 2025, https://www.usenix.org/conference/atc25/presentation/xie, pp. 1425–1440.